

ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

FLIGHT TICKET BOOKING

Network Programming: Application Protocol Design

Presented by:

Flight Ticket Project

Design review 6 November 2026

ONE LOVE. ONE FUTURE.

Application problem and scope

Customer goal

- Search several airlines and compare price and flight duration.
- Direct book and pay from balance, receive ticket codes.
- Manage bookings: cancel with 100% refund, change, print.
- Receive flight updates even after reconnecting.

Project scope

- Mock domestic flights and account-balance payment.
- Customer and admin clients use direct sockets.
- Real e-ticket email only. SMS is excluded.

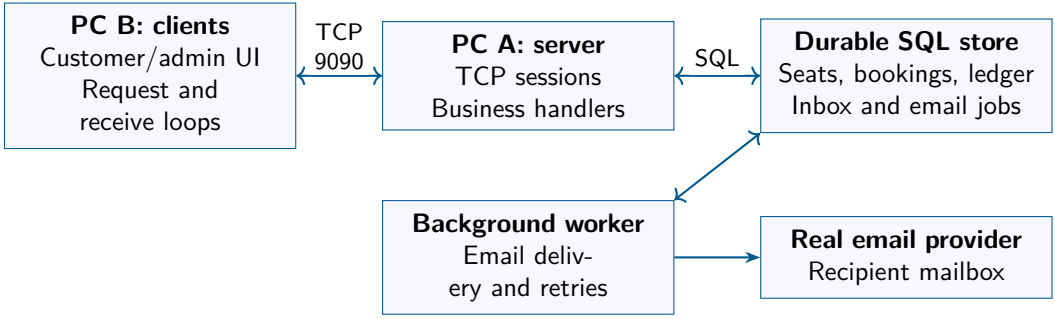
Design objective: keep seats, booking status, balance and ticket records consistent when requests fail or connections drop.

Required functionality and grading coverage

Requirement	Observable application behavior	Points
R1 Sockets	Client and server communicate over a custom socket protocol	2
R2 Logging	Server records received and sent messages	1
R3 Accounts	Register, authenticate and log out	2
R4 Search	Route, departure/return date, passengers and seat class	2
R5 Compare	Airline prices and flight duration	2
R6 Purchase	Direct book and pay: check balance and issue tickets	3
R7 E-ticket	Send ticket codes through real email	2
R8 Management	View, cancel (with refund), change and print tickets	3
R9 Notifications	Admin triggers schedule, cancellation and delay updates	3

Total: 20 points. Protocol design supports every function.

Client–server architecture



Server: C, C++ or Java with direct socket handling. Multiple clients connect concurrently. Server logs track receipt, rejection and send outcomes.

Customer and admin procedures

Actor	Procedure	Client/server responsibility
Guest	Register, then log in	Server creates USER and binds identity to the connection
Customer	Search, compare, book & pay	Client submits selection. Server checks ticket status and balance, debits, issues tickets
Customer	Manage tickets, cancel $\rightarrow$ refund	Server processes change/print; verifies ≥ 2 h cutoff, cancels tickets and refunds 100% fare
Customer	Review flight notifications	Client handles pushes and retrieves unread inbox entries
Admin	List flights, delay, reschedule, cancel	Server checks ADMIN, updates affected bookings and saves notifications

Access rule: customers see only their own bookings and notifications. Registration can never create an admin.

Search: HAN to SGN, 20 December 2026, one adult, ECONOMY.

Flight	Airline	Fare (VND)	Duration
F1002 / VJ131	VietJet Air	1,290,000	130 min
F1003 / QH201	Bamboo Airways	1,590,000	125 min
F1001 / VN213	Vietnam Airlines	1,850,000	130 min

- Search also accepts return date, passenger count, class and sorting.
- Compare 2–5 flights on the same route/date/class. Mark cheapest and fastest.
- Exclude unavailable flights or insufficient seats. Search and comparison hold no seats.

TCP framing and transport rules

One message: one UTF-8 JSON object followed by LF.

```
{"v":1,"type":"PING","id":"p1","data":{}}\n
```

Condition	Required receiver/sender behavior
Partial read	Keep bytes until LF completes the frame
Several frames in one read	Process complete lines in order, retain the unfinished suffix
Partial write	Continue until the entire frame and LF have been sent
Frame limit	At most 65,536 payload bytes, excluding LF and optional CR
Timing	Incomplete frame: 10 s. Idle receive: 300 s. Client request: 10 s

One client request may wait at a time. Serialize complete outgoing frames.

Request and matching response

```
{"v":1,"type":"PING","id":"p1","data":{}}
{"v":1,"type":"PING_RESP","id":"p1","ok":true,
 "code":"OK","data":{"serverTime":"2026-11-04T10:00:00+07:00"}}
```

Failure reply

```
{"v":1,"type":"LOGIN_RESP","id":"l1","ok":false,
 "code":"E_BAD_CREDENTIALS","message":"Invalid credentials","data":{}}
```

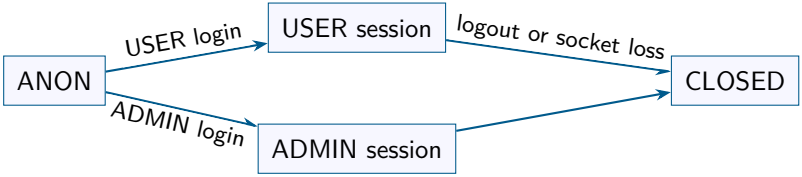
- `v`: protocol version. `type`: operation. `id`: request correlation.
- `NOTIFY` uses `id:null` and has no `ok` field.
- Clients dispatch by type/ID. A push does not complete a pending request.

Protocol operation inventory

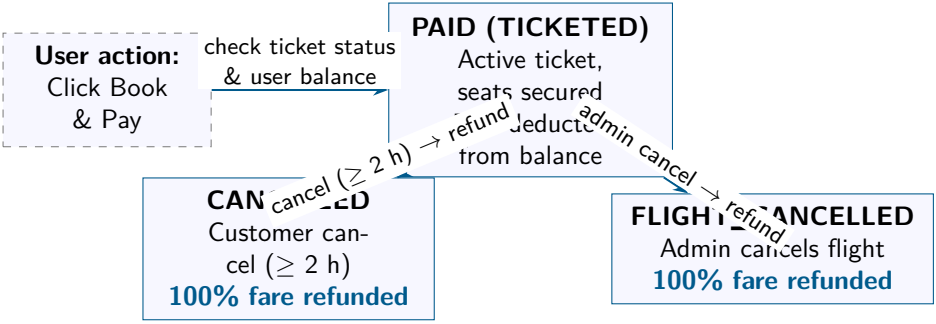
Group	Request types
Session and account	`PING`, `REGISTER`, `LOGIN`, `LOGOUT`, `GET_BALANCE`
Search and comparison	`SEARCH_FLIGHTS`, `COMPARE_FLIGHTS`
Booking and payment	`BOOK_AND_PAY`, `RESEND_TICKET`
Ticket management	`LIST_BOOKINGS`, `GET_BOOKING`, `CANCEL_BOOKING`, `CHANGE_BOOKING`, `PRINT_TICKET`
Notifications	`GET_NOTIFICATIONS`, `ACK_NOTIFICATION`
Admin	`ADMIN_LIST_FLIGHTS`, `ADMIN_UPDATE_FLIGHT`

18 request types, typed responses and one unsolicited server message: `NOTIFY`.

Session and client request states

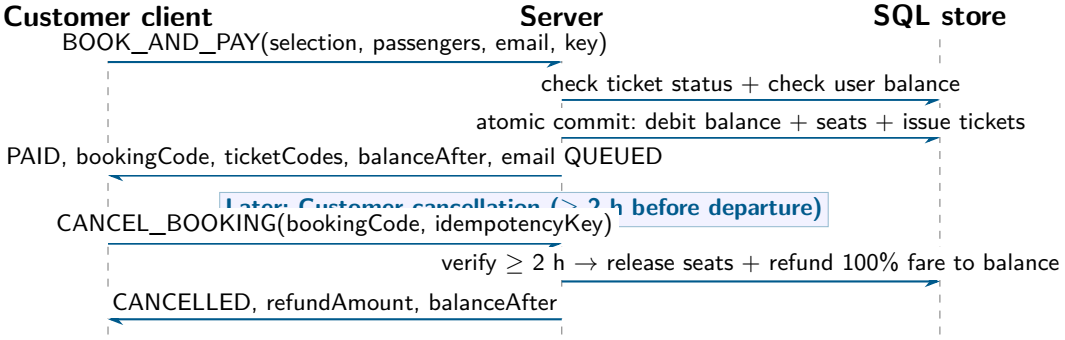


- REGISTER and failed LOGIN leave the connection ANON.
- Role and identity come from the server's stored account, never request fields.
- Client request states: READY, WAITING and UNKNOWN OUTCOME.
- During WAITING, handle `NOTIFY` and continue waiting for the matching response.
- Reconnect starts ANON. Authenticate again before recovering an uncertain operation.



Single-step book and pay: User clicks book and pay → server checks ticket status and user balance → deducts balance and creates `PAID` booking. If checks fail, returns error (zero deduction).
Later cancellation → refund: Customer cancel ≥ 2 h (or flight disruption/admin cancel) releases seats and refunds 100% fare back to user account balance.

Booking, payment and cancellation sequence



Direct flow: User clicks book and pay → server checks ticket status and user balance → deducts balance and creates `PAID` ticket. Later cancellation ($\geq$ 2 h) refunds 100% fare to user balance.

Real e-ticket email delivery

Stage	Behavior
Book & pay commits	Booking is PAID. Store ticket codes and an email job
Worker sends	Use the saved booking contactEmail and a configured real provider
Delivery status	QUEUED, SENT on provider acceptance, or FAILED
Provider fails	Keep the paid booking. Retry the email job without another charge
Customer resends	`RESEND_TICKET(bookingCode)` uses the same saved recipient/codes

- Email only. Provider credentials remain on the server.
- A timeout may duplicate email on retry, never payment or tickets.
- R7 demonstration must show actual inbox receipt with matching booking/ticket codes.

Cancellation and simple refund rules

Cancellation scenario	**Refund and booking outcome**
Customer cancel $\geq$ 2 h before departure	Allow cancellation. **Refund 100% fare** to account balance
Customer cancel $<$ 2 h (or departed)	Reject with `E_CANCEL_NOT_ALLOWED`. Ticket remains active
Disrupted flight (delayed $\geq$ 2 h / rescheduled)	Allow cancellation. **Refund 100% fare** to balance, even within 2 h
Admin cancels flight	Automatic cancellation. **Refund 100% fare** to all affected bookings

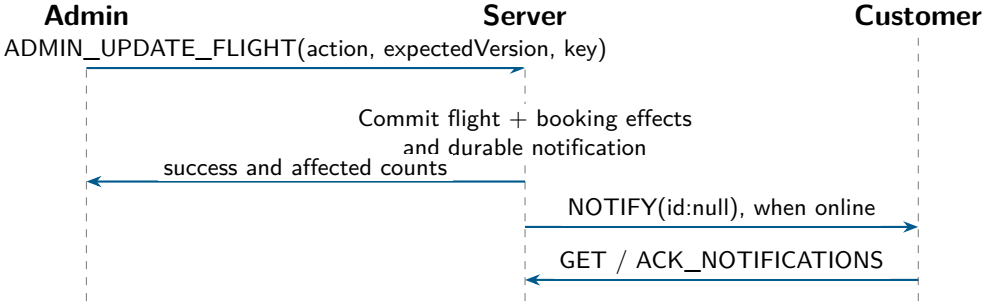
- **100% direct refund:** Eligible cancellations refund full fare directly to user account balance; cancellation fee is zero.
- **Simple protocol:** Client sends `CANCEL_BOOKING(bookingCode, idempotencyKey)`. Server verifies cutoff and processes refund atomically.

Viewing, changing and printing tickets

Operation	Procedure and outcome
View	LIST_BOOKINGS, then GET_BOOKING for current itinerary, status and tickets
Change	Select replacement on the same route/class. Preview a 60-second quote, then apply with quoteId/key
Settle change	Proposed fee 200,000 VND/passenger, waived if disrupted, plus signed fare difference
Update ticket	Swap seats atomically, increment revision and queue revised email. Preserve booking/ticket identifiers
Print	PRINT_TICKET returns current text lines. Client opens its print/save dialog

Change requires every current leg and replacement to depart at least 2 h later. Any failed change preserves the original itinerary and tickets.

Admin updates and customer notifications



DELAY/RESCHEDULE updates paid itineraries and notifies customers. CANCEL cancels affected flights, releases seats, and refunds paid bookings once. Offline customers recover from the saved inbox.

Message processing on both ends

Server processing

1. Assemble and parse a bounded frame.
2. Log safely. Validate envelope, fields, role and ownership.
3. Check replay and business guards.
4. Commit inventory, balance, booking and saved result together.
5. Enqueue reply/push and write all bytes.

Client processing

1. Validate input and retain request ID/key.
2. Send one request and keep receiving.
3. Dispatch NOTIFY separately from replies.
4. Match response ID/type and show authoritative results/errors.
5. On timeout, reconnect, log in and reconcile.

One domain lock plus a SQL transaction serializes small-demo mutations. Socket writes and provider calls happen after release.

Failure class	Response and recovery
Invalid complete JSON frame	ERROR_RESP / E_BAD_JSON. Keep the connection usable
Missing login or wrong role	E_NOT_AUTHENTICATED or E_FORBIDDEN. No business mutation
Business rejection	Typed error, such as E_SEATS_UNAVAILABLE or E_INSUFFICIENT_BALANCE
Oversize / incomplete-frame timeout	Bound memory, log the fault and close that connection

- Log timestamp, peer/connection, user, direction, type/ID, byte count and result.
- Distinguish SEND_ATTEMPT, SEND_SUCCESS and SEND_FAILURE.
- Redact passwords, contacts and passenger identity data. Never log malformed raw payloads.

Recovery after a lost reply

Example: book and pay commits, but the connection closes before the reply arrives.

1. Client records an unknown outcome and keeps the original payload/key.
2. Reconnect and log in again with a fresh connection request ID.
3. Retry the same mutation with its original idempotencyKey.
4. Server returns the saved success before checking the current booking state.
5. Client refreshes the booking, balance and unread notifications.

Request ID	Correlates one exchange on one connection
Idempotency key	Identifies a committed operation across reconnect/restart



THANK YOU !